



IT SIMPLERA INSTITUTE INTERNSHIP PROGRAM

WEEK 04 REPORT

Advanced Malware Analysis and Incident Response

Submitted By

Name: Challa Madhumitha

Reg No: SOCB01-2121

Role: Soc Analyst Intern

Submission Date: 26th July 2026

Instructor: Zia Ullah

TABLE OF CONTENTS

S.No	Contents	Page No.
1	Introduction	3
2	Objective	3
3	Lab Environment	3
4	Malware Sample Preparation	4
5	Static Analysis	5
6	Dynamic Analysis	8
7	Indicators of Compromise (IOCs)	11
8	Hybrid Analysis	11
9	Comparison with Public Malware Reports	12
10	Incident Response Plan	12
11	Results	13
12	Learning Outcomes	14
13	Conclusion	14

1. Introduction

Malware analysis is the process of examining malicious software to understand its functionality, behavior, persistence mechanisms, communication methods, and overall impact on a system. It enables security professionals to identify Indicators of Compromise (IOCs), improve detection capabilities, and develop effective response strategies.

Security Operations Centers (SOCs) rely on malware analysis to investigate suspicious files, understand attacker techniques, and improve organizational defenses. Malware analysis generally consists of two major approaches:

- Static Analysis, where the malware is examined without executing it.
- Dynamic Analysis, where the malware is executed in a controlled and isolated environment to observe its behavior.

In this project, a previously documented malware sample was analyzed using both static and dynamic analysis techniques. The collected evidence was compared with publicly available malware intelligence reports, and an Incident Response Plan was prepared based on the observed malicious activities.

2. Objectives

The objectives of this project were:

- Prepare a safe malware analysis laboratory.
- Download a previously reported malware sample from a trusted source.
- Store the malware inside a password-protected ZIP archive.
- Perform static malware analysis.
- Perform dynamic malware analysis inside an isolated virtual machine.
- Monitor process creation, registry modifications, network and file system activities.
- Identify Indicators of Compromise (IOCs).
- Compare findings with public malware reports.
- Develop a complete Incident Response Plan.

3. Lab Environment

A secure virtual environment was prepared to safely analyze malware without affecting the host operating system. All analysis activities were performed inside isolated virtual machines.

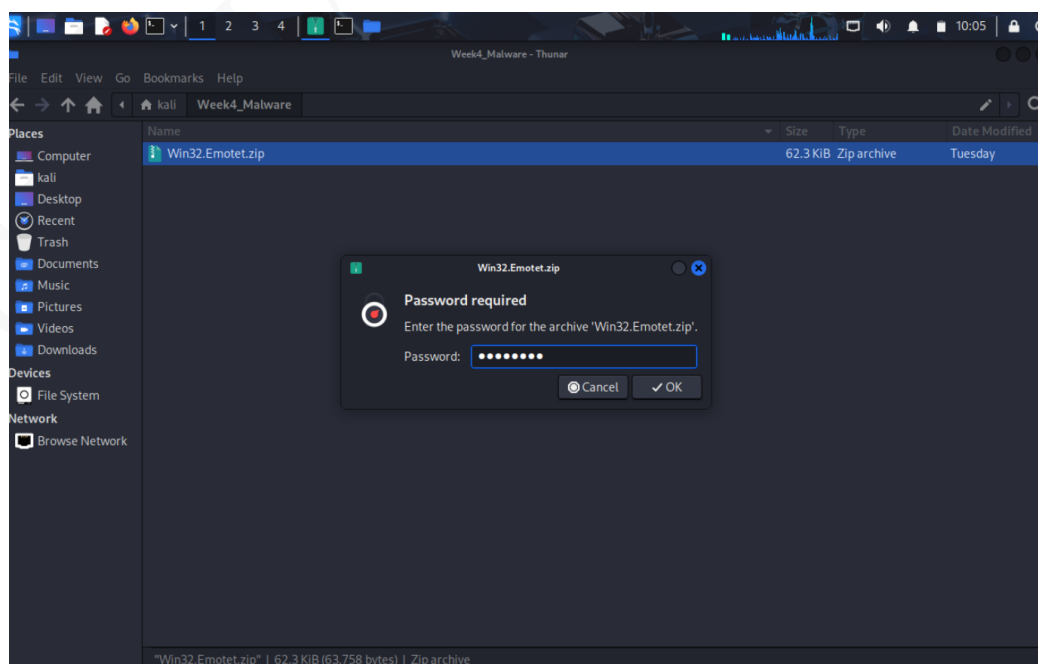
Software Used

Software	Purpose
VMware Workstation	Virtualization Platform
Windows 10 VM	Malware Analysis Machine
Kali Linux	Security Tools
Process Explorer	Process Monitoring
Process Monitor (Procmon)	File & Registry Monitoring
Wireshark	Packet Capture
PEStudio	Static Analysis
FLOSS / Strings	String Extraction
VirusTotal	Malware Reputation Comparison
Wazuh SIEM	Security Monitoring

4. Malware Sample Preparation

A previously reported malware sample was obtained from a trusted malware research source. To prevent accidental execution, the sample was stored inside a password-protected ZIP archive. Password protection is a standard practice in malware research because it prevents antivirus software from automatically extracting or executing the sample.

Before analysis, cryptographic hashes (MD5 and SHA-256) were generated to uniquely identify the malware sample and ensure integrity throughout the investigation.



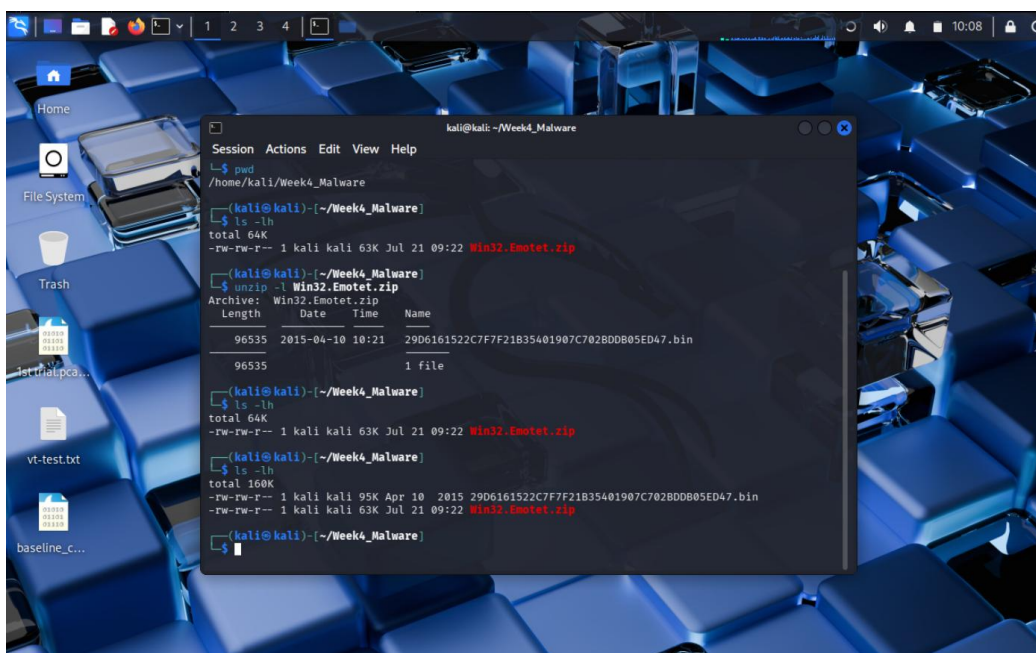


Figure 1 Password-Protected Malware Sample

5. Static Malware Analysis

Static analysis was performed without executing the malware sample. This approach helps analysts understand the structure and characteristics of the executable while minimizing risk.

The following information was collected:

- File Name
- File Size
- MD5 Hash
- SHA-256 Hash
- PE Header Information
- Compilation Timestamp
- Imported DLLs
- Exported Functions
- Embedded Strings
- File Entropy
- Digital Signature

PEStudio was used to examine the executable structure, while FLOSS and the Strings utility extracted human-readable strings embedded within the malware. These strings revealed

possible command-and-control domains, file paths, registry keys, API calls, and suspicious messages.

The top screenshot shows a Kali Linux terminal window with the following commands and output:

```

kali@kali: ~/Week4_Malware
$ ls -lh
total 160K
-rw-rw-r-- 1 kali kali 95K Apr 10 2015 29D6161522C7F7F21B35401907C702BDD805ED47.bin
-rw-rw-r-- 1 kali kali 63K Jul 21 09:22 Min32.Emotet.zip

(kali@kali)~/Week4_Malware
$ file 29D6161522C7F7F21B35401907C702BDD805ED47.bin
29D6161522C7F7F21B35401907C702BDD805ED47.bin: PE32 executable for MS Windows 4.00 (GUI), Intel i
386, 3 sections

(kali@kali)~/Week4_Malware
$ ls -lh 29D6161522C7F7F21B35401907C702BDD805ED47.bin
-rw-rw-r-- 1 kali kali 95K Apr 10 2015 29D6161522C7F7F21B35401907C702BDD805ED47.bin

(kali@kali)~/Week4_Malware
$ md5sum 29D6161522C7F7F21B35401907C702BDD805ED47.bin
8baa9b09b591a11af423824f4d9726a 29D6161522C7F7F21B35401907C702BDD805ED47.bin

(kali@kali)~/Week4_Malware
$ sha256sum 29D6161522C7F7F21B35401907C702BDD805ED47.bin
6393Fe8dd4721190f240e22feb769675b6194a70cabd5a415c2364686a9089c 29D6161522C7F7F21B35401907C702
BDD805ED47.bin

(kali@kali)~/Week4_Malware
$ obfdump -x 29D6161522C7F7F21B35401907C702BDD805ED47.bin | less

(kali@kali)~/Week4_Malware
$ obfdump -x 29D6161522C7F7F21B35401907C702BDD805ED47.bin | less

(kali@kali)~/Week4_Malware
$ obfdump -p 29D6161522C7F7F21B35401907C702BDD805ED47.bin | grep "DLL Name"
DLL Name: MSVBVM60.DLL

(kali@kali)~/Week4_Malware

```

The bottom screenshot shows the PESTudio interface with the following details:

File: 29D6161522C7F7F21B35401907C702BDD805ED47.bin
 Architecture: i386, flags: 0x00001000
 Entry: 0x00401000, OS: Win32
 Start address: 0x00401000

Characteristics: 0x00000000
 Relocations: stripped
 Executable: yes
 Line numbers: stripped
 Symbols: stripped
 32 bit words

Time/Date: Tue Mar 31 04:53:51 2015
 Magic: 0x00000000 (PE32)
 MajorLinkerVersion: 8
 MinorLinkerVersion: 0
 SizeOfCode: 0x00070000
 SizeOfInitializedData: 0x00000000
 SizeOfUninitializedData: 0x00000000
 AddressOfEntryPoint: 0x00401000
 BaseOfCode: 0x00000000
 BaseOfData: 0x00000000
 ImageBase: 0x00400000
 SectionAlignment: 0x00000000
 FileAlignment: 0x00000000
 MajorSystemVersion: 4
 MinorSystemVersion: 0
 MajorImageVersion: 1
 MinorImageVersion: 0
 MajorSubsystemVersion: 4
 MinorSubsystemVersion: 0
 Win32Version: 0x00000000
 SizeOfFlags: 0x00000000
 SizeOfHeaders: 0x00000000
 CheckSum: 0x00000000
 Subsystem: 3 (Windows GUI)
 DllCharacteristics: 0x00000000
 SizeOfStackReserve: 0x00000000
 SizeOfStackCommit: 0x00000000
 SizeOfHeapReserve: 0x00000000
 SizeOfHeapCommit: 0x00000000
 LoaderFlags: 0x00000000
 NumberOfVaListProcs: 0x00000000

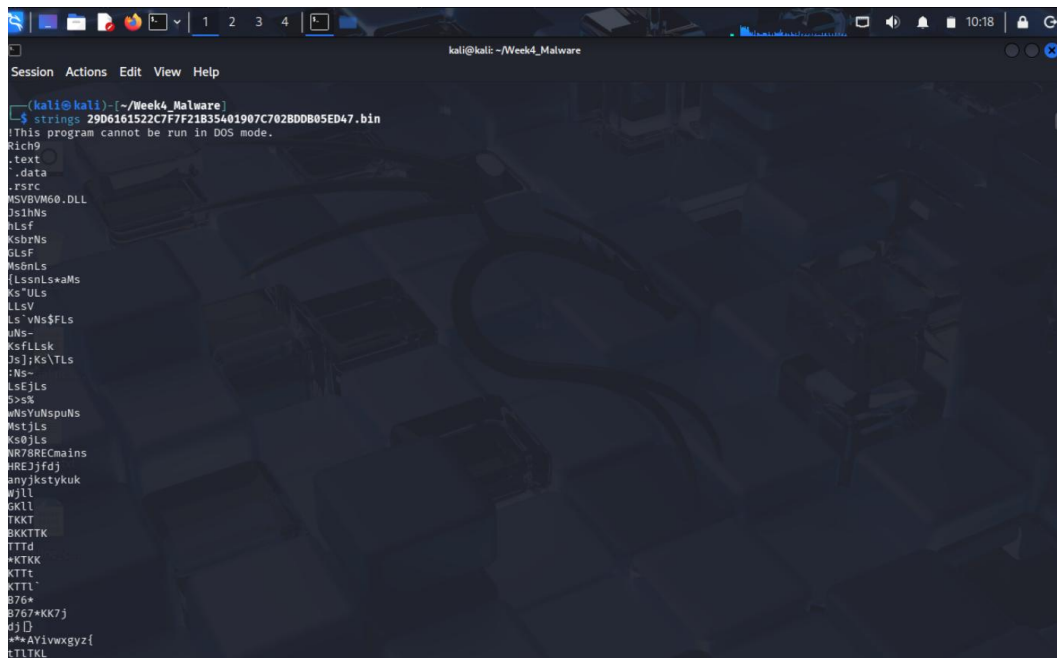
The Data Directory:

Entry	Virtual Address	Size	Description
Export Directory	0x00000000	0x00000000	Export Directory (.edata) (or where ever we found it)
Import Directory	0x00000000	0x00000000	Import Directory (.idata)
Resource Directory	0x00000000	0x00000000	Resource Directory (.rsrc)
Exception Directory	0x00000000	0x00000000	Exception Directory (.pdata)
Security Directory	0x00000000	0x00000000	Security Directory (.security)
Base Relocation Directory	0x00000000	0x00000000	Base Relocation Directory (.reloc)
Debug Directory	0x00000000	0x00000000	Debug Directory (.debug)
String Directory	0x00000000	0x00000000	String Directory (.string)
Special Directory	0x00000000	0x00000000	Special Directory (.special)
Thread Storage Directory	0x00000000	0x00000000	Thread Storage Directory (.tls)
Load Configuration Directory	0x00000000	0x00000000	Load Configuration Directory (.load_config)
Bound Image Directory	0x00000000	0x00000000	Bound Image Directory (.bind_image)
Import Address Table Directory	0x00000000	0x00000000	Import Address Table Directory (.import_addr_table)
Delay Import Directory	0x00000000	0x00000000	Delay Import Directory (.delay_import)
CLR Header	0x00000000	0x00000000	CLR Header

There is an import table in .text at 0x00401000

The Import Tables (interpreted .text section contents):

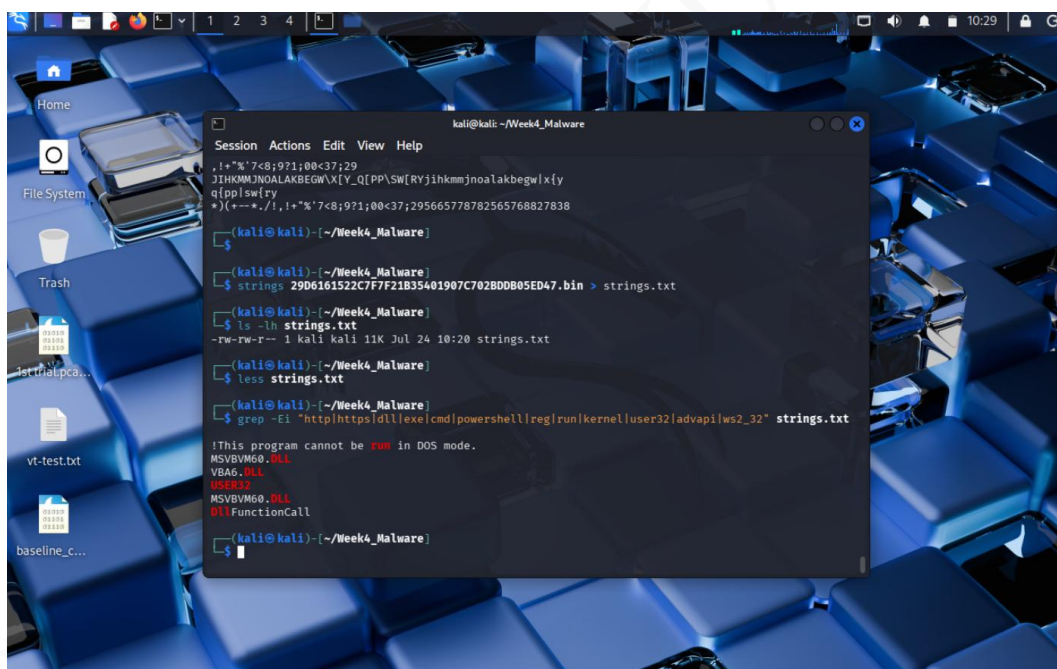
Ordinal	Module Name	Import Name	Import Address
0x00000000	kernel32.dll	GetModuleHandleA	0x00401000
0x00000001	kernel32.dll	ExitProcess	0x00401000
0x00000002	kernel32.dll	FreeLibrary	0x00401000
0x00000003	kernel32.dll	LoadLibraryA	0x00401000
0x00000004	kernel32.dll	GetProcAddress	0x00401000
0x00000005	kernel32.dll	SetFileAttributesA	0x00401000
0x00000006	kernel32.dll	GetFileAttributesA	0x00401000
0x00000007	kernel32.dll	FindFileA	0x00401000
0x00000008	kernel32.dll	FindFirstFileA	0x00401000
0x00000009	kernel32.dll	FindNextFileA	0x00401000
0x0000000A	kernel32.dll	FindFileExA	0x00401000
0x0000000B	kernel32.dll	FindFirstFileExA	0x00401000
0x0000000C	kernel32.dll	FindNextFileExA	0x00401000
0x0000000D	kernel32.dll	FindFileExInfoA	0x00401000
0x0000000E	kernel32.dll	FindFirstFileExInfoA	0x00401000
0x0000000F	kernel32.dll	FindNextFileExInfoA	0x00401000
0x00000010	kernel32.dll	FindFileExInfoA	0x00401000
0x00000011	kernel32.dll	FindFirstFileExInfoA	0x00401000
0x00000012	kernel32.dll	FindNextFileExInfoA	0x00401000
0x00000013	kernel32.dll	FindFileExInfoA	0x00401000
0x00000014	kernel32.dll	FindFirstFileExInfoA	0x00401000
0x00000015	kernel32.dll	FindNextFileExInfoA	0x00401000
0x00000016	kernel32.dll	FindFileExInfoA	0x00401000
0x00000017	kernel32.dll	FindFirstFileExInfoA	0x00401000
0x00000018	kernel32.dll	FindNextFileExInfoA	0x00401000
0x00000019	kernel32.dll	FindFileExInfoA	0x00401000
0x0000001A	kernel32.dll	FindFirstFileExInfoA	0x00401000
0x0000001B	kernel32.dll	FindNextFileExInfoA	0x00401000
0x0000001C	kernel32.dll	FindFileExInfoA	0x00401000
0x0000001D	kernel32.dll	FindFirstFileExInfoA	0x00401000
0x0000001E	kernel32.dll	FindNextFileExInfoA	0x00401000
0x0000001F	kernel32.dll	FindFileExInfoA	0x00401000
0x00000020	kernel32.dll	FindFirstFileExInfoA	0x00401000
0x00000021	kernel32.dll	FindNextFileExInfoA	0x00401000
0x00000022	kernel32.dll	FindFileExInfoA	0x00401000
0x00000023	kernel32.dll	FindFirstFileExInfoA	0x00401000
0x00000024	kernel32.dll	FindNextFileExInfoA	0x00401000
0x00000025	kernel32.dll	FindFileExInfoA	0x00401000
0x00000026	kernel32.dll	FindFirstFileExInfoA	0x00401000
0x00000027	kernel32.dll	FindNextFileExInfoA	0x00401000
0x00000028	kernel32.dll	FindFileExInfoA	0x00401000
0x00000029	kernel32.dll	FindFirstFileExInfoA	0x00401000
0x0000002A	kernel32.dll	FindNextFileExInfoA	0x00401000
0x0000002B	kernel32.dll	FindFileExInfoA	0x00401000
0x0000002C	kernel32.dll	FindFirstFileExInfoA	0x00401000
0x0000002D	kernel32.dll	FindNextFileExInfoA	0x00401000
0x0000002E	kernel32.dll	FindFileExInfoA	0x00401000
0x0000002F	kernel32.dll	FindFirstFileExInfoA	0x00401000
0x00000030	kernel32.dll	FindNextFileExInfoA	0x00401000
0x00000031	kernel32.dll	FindFileExInfoA	0x00401000
0x00000032	kernel32.dll	FindFirstFileExInfoA	0x00401000
0x00000033	kernel32.dll	FindNextFileExInfoA	0x00401000
0x00000034	kernel32.dll	FindFileExInfoA	0x00401000
0x00000035	kernel32.dll	FindFirstFileExInfoA	0x00401000
0x00000036	kernel32.dll	FindNextFileExInfoA	0x00401000
0x00000037	kernel32.dll	FindFileExInfoA	0x00401000
0x00000038	kernel32.dll	FindFirstFileExInfoA	0x00401000
0x00000039	kernel32.dll	FindNextFileExInfoA	0x00401000
0x0000003A	kernel32.dll	FindFileExInfoA	0x00401000
0x0000003B	kernel32.dll	FindFirstFileExInfoA	0x00401000
0x0000003C	kernel32.dll	FindNextFileExInfoA	0x00401000
0x0000003D	kernel32.dll	FindFileExInfoA	0x00401000
0x0000003E	kernel32.dll	FindFirstFileExInfoA	0x00401000
0x0000003F	kernel32.dll	FindNextFileExInfoA	0x00401000
0x00000040	kernel32.dll	FindFileExInfoA	0x00401000
0x00000041	kernel32.dll	FindFirstFileExInfoA	0x00401000
0x00000042	kernel32.dll	FindNextFileExInfoA	0x00401000
0x00000043	kernel32.dll	FindFileExInfoA	0x00401000
0x00000044	kernel32.dll	FindFirstFileExInfoA	0x00401000
0x00000045	kernel32.dll	FindNextFileExInfoA	0x00401000
0x00000046	kernel32.dll	FindFileExInfoA	0x00401000
0x00000047	kernel32.dll	FindFirstFileExInfoA	0x00401000
0x00000048	kernel32.dll	FindNextFileExInfoA	0x00401000
0x00000049	kernel32.dll	FindFileExInfoA	0x00401000
0x0000004A	kernel32.dll	FindFirstFileExInfoA	0x00401000
0x0000004B	kernel32.dll	FindNextFileExInfoA	0x00401000
0x0000004C	kernel32.dll	FindFileExInfoA	0x00401000
0x0000004D	kernel32.dll	FindFirstFileExInfoA	0x00401000
0x0000004E	kernel32.dll	FindNextFileExInfoA	0x00401000
0x0000004F	kernel32.dll	FindFileExInfoA	0x00401000
0x00000050	kernel32.dll	FindFirstFileExInfoA	0x00401000
0x00000051	kernel32.dll	FindNextFileExInfoA	0x00401000
0x00000052	kernel32.dll	FindFileExInfoA	0x00401000
0x00000053	kernel32.dll	FindFirstFileExInfoA	0x00401000
0x00000054	kernel32.dll	FindNextFileExInfoA	0x00401000
0x00000055	kernel32.dll	FindFileExInfoA	0x00401000
0x00000056	kernel32.dll	FindFirstFileExInfoA	0x00401000
0x00000057	kernel32.dll	FindNextFileExInfoA	0x00401000
0x00000058	kernel32.dll	FindFileExInfoA	0x00401000
0x00000059	kernel32.dll	FindFirstFileExInfoA	0x00401000
0x0000005A	kernel32.dll	FindNextFileExInfoA	0x00401000
0x0000005B	kernel32.dll	FindFileExInfoA	0x00401000
0x0000005C	kernel32.dll	FindFirstFileExInfoA	0x00401000
0x0000005D	kernel32.dll	FindNextFileExInfoA	0x00401000
0x0000005E	kernel32.dll	FindFileExInfoA	0x00401000
0x0000005F	kernel32.dll	FindFirstFileExInfoA	0x00401000
0x00000060	kernel32.dll	FindNextFileExInfoA	0x00401000
0x00000061	kernel32.dll	FindFileExInfoA	0x00401000
0x00000062	kernel32.dll	FindFirstFileExInfoA	0x00401000
0x00000063	kernel32.dll	FindNextFileExInfoA	0x00401000
0x00000064	kernel32.dll	FindFileExInfoA	0x00401000
0x00000065	kernel32.dll	FindFirstFileExInfoA	0x00401000
0x00000066	kernel32.dll	FindNextFileExInfoA	0x00401000
0x00000067	kernel32.dll	FindFileExInfoA	0x00401000
0x00000068	kernel32.dll	FindFirstFileExInfoA	0x00401000
0x00000069	kernel32.dll	FindNextFileExInfoA	0x00401000
0x0000006A	kernel32.dll	FindFileExInfoA	0x00401000
0x0000006B	kernel32.dll	FindFirstFileExInfoA	0x00401000
0x0000006C	kernel32.dll	FindNextFileExInfoA	0x00401000
0x0000006D	kernel32.dll	FindFileExInfoA	0x00401000
0x0000006E	kernel32.dll	FindFirstFileExInfoA	0x00401000
0x0000006F	kernel32.dll	FindNextFileExInfoA	0x00401000
0x00000070	kernel32.dll	FindFileExInfoA	0x00401000
0x00000071	kernel32.dll	FindFirstFileExInfoA	0x00401000
0x00000072	kernel32.dll	FindNextFileExInfoA	0x00401000
0x00000073	kernel32.dll	FindFileExInfoA	0x00401000
0x00000074	kernel32.dll	FindFirstFileExInfoA	0x00401000
0x00000075	kernel32.dll	FindNextFileExInfoA	0x00401000
0x00000076	kernel32.dll	FindFileExInfoA	0x00401000
0x00000077	kernel32.dll	FindFirstFileExInfoA	0x00401000
0x00000078	kernel32.dll	FindNextFileExInfoA	0x00401000
0x00000079	kernel32.dll	FindFileExInfoA	0x00401000
0x0000007A	kernel32.dll	FindFirstFileExInfoA	0x00401000
0x0000007B	kernel32.dll	FindNextFileExInfoA	0x00401000
0x0000007C	kernel32.dll	FindFileExInfoA	0x00401000
0x0000007D	kernel32.dll	FindFirstFileExInfoA	0x00401000
0x0000007E	kernel32.dll	FindNextFileExInfoA	0x00401000
0x0000007F	kernel32.dll	FindFileExInfoA	0x00401000
0x00000080	kernel32.dll	FindFirstFileExInfoA	0x00401000
0x00000081	kernel32.dll	FindNextFileExInfoA	0x00401000
0x00000082	kernel32.dll	FindFileExInfoA	0x00401000
0x00000083	kernel32.dll	FindFirstFileExInfoA	0x00401000
0x00000084	kernel32.dll	FindNextFileExInfoA	0x00401000
0x00000085	kernel32.dll	FindFileExInfoA	0x00401000
0x00000086	kernel32.dll	FindFirstFileExInfoA	0x00401000
0x00000087	kernel32.dll	FindNextFileExInfoA	0x00401000
0x00000088	kernel32.dll	FindFileExInfoA	0x00401000
0x00000089	kernel32.dll	FindFirstFileExInfoA	0x00401000
0x0000008A	kernel32.dll	FindNextFileExInfoA	0x00401000
0x0000008B	kernel32.dll	FindFileExInfoA	0x00401000
0x0000008C	kernel32.dll	FindFirstFileExInfoA	0x00401000
0x0000008D	kernel32.dll	FindNextFileExInfoA	0x00401000
0x0000008E	kernel32.dll	FindFileExInfoA	0x00401000
0x0000008F	kernel32.dll	FindFirstFileExInfoA	0x00401000
0x00000090	kernel32.dll	FindNextFileExInfoA	0x00401000
0x00000091	kernel32.dll	FindFileExInfoA	0x00401000
0x00000092	kernel32.dll	FindFirstFileExInfoA	0x00401000
0x00000093	kernel32.dll	FindNextFileExInfoA	0x00401000
0x00000094	kernel32.dll	FindFileExInfoA	0x00401000
0x00000095	kernel32.dll	FindFirstFileExInfoA	0x00401000
0x00000096	kernel32.dll	FindNextFileExInfoA	0x00401000
0x00000097	kernel32.dll	FindFileExInfoA	0x00401000
0x00000098	kernel32.dll	FindFirstFileExInfoA	0x00401000
0x00000099	kernel32.dll	FindNextFileExInfoA	0x00401000
0x0000009A	kernel32.dll	FindFileExInfoA	0x00401000
0x0000009B	kernel32.dll	FindFirstFileExInfoA	0x00401000
0x0000009C	kernel32.dll	FindNextFileExInfoA	0x00401000
0x0000009D	kernel32.dll	FindFileExInfoA	0x00401000



```

kali@kali: ~/Week4_Malware
Session Actions Edit View Help
kali@kali:~/Week4_Malware$ strings 29D6161522C7F7F21B35401907C7028D0B05ED47.bin
!This program cannot be run in DOS mode.
Rich9
.text
.data
.rsrc
MSVBVM60.DLL
Is1hNs
hLsf
KsbrNs
GLSF
R6gnLs
(LssnLsaaMs
s"ULs
LLsV
Ls"vNs$FLs
uNs-
KsflLsk
Isj;Ks\Tls
iNs-
LsEjLs
b>s%
uNsYuNspuNs
KstjLs
Ks0jLs
NR78REcmaInS
HREjJfdj
anyjKstyKuk
Kjll
Kkll
TKKT
BKKTtK
TTTd
*KTtK
KTtT
KTtL
576*
B767*KK7j
Kj[]
**AYIwxgyz{
tLTkL

```



```

kali@kali: ~/Week4_Malware
Session Actions Edit View Help
kali@kali:~/Week4_Malware$ strings 29D6161522C7F7F21B35401907C7028D0B05ED47.bin > strings.txt
kali@kali:~/Week4_Malware$ ls -lh strings.txt
-rw-rw-r-- 1 kali kali 11K Jul 24 10:20 strings.txt
kali@kali:~/Week4_Malware$ less strings.txt
kali@kali:~/Week4_Malware$ grep -Ei "http|https|dll|exe|cmd|powershell|reg|run|kernel|user32|advapi|ws2_32" strings.txt
!This program cannot be run in DOS mode.
MSVBVM60.DLL
VBAG.DLL
Uo8s2p
MSVBVM60.DLL
DllFunctionCall

```

Figure 3 Extracted Strings

6. Dynamic Malware Analysis

Dynamic analysis was performed by executing the malware inside an isolated virtual machine while monitoring its behavior using forensic tools.

The following activities were monitored:

Process Activity

Process Explorer was used to observe process creation, parent-child relationships, process injection attempts, and CPU usage.

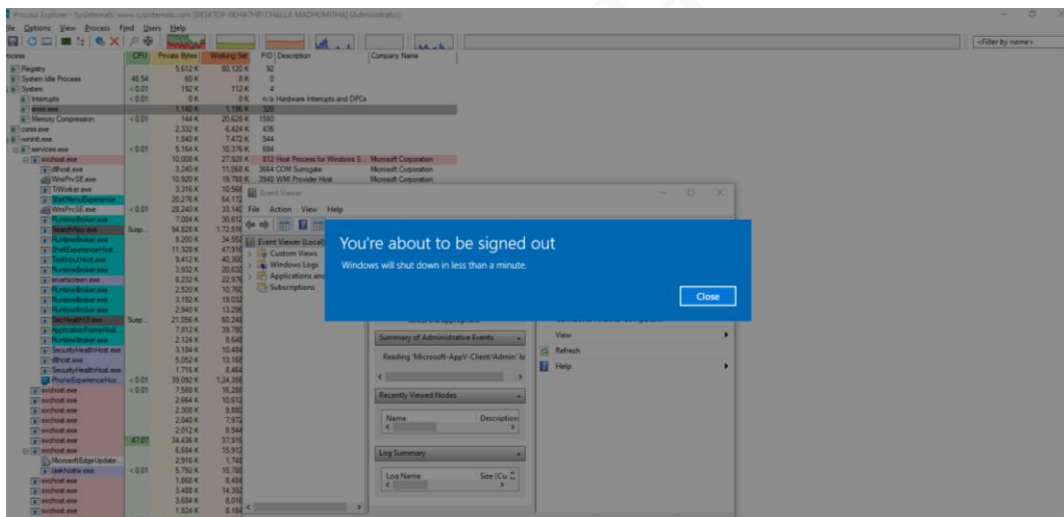
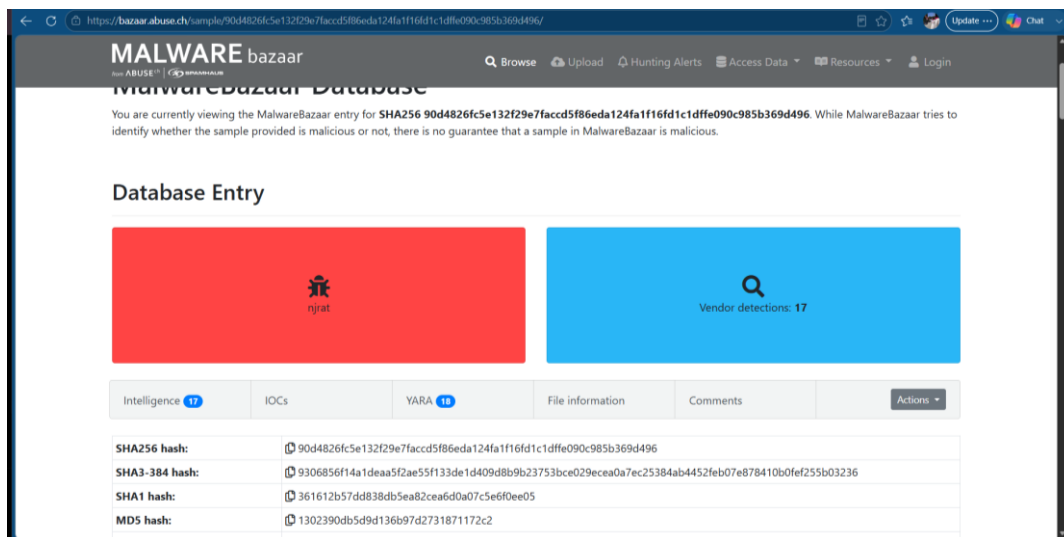


Figure 4 Process Explorer showing malware execution

Registry Activity

Process Monitor recorded registry modifications performed by the malware. Registry keys used for persistence, startup entries, and configuration changes were identified.

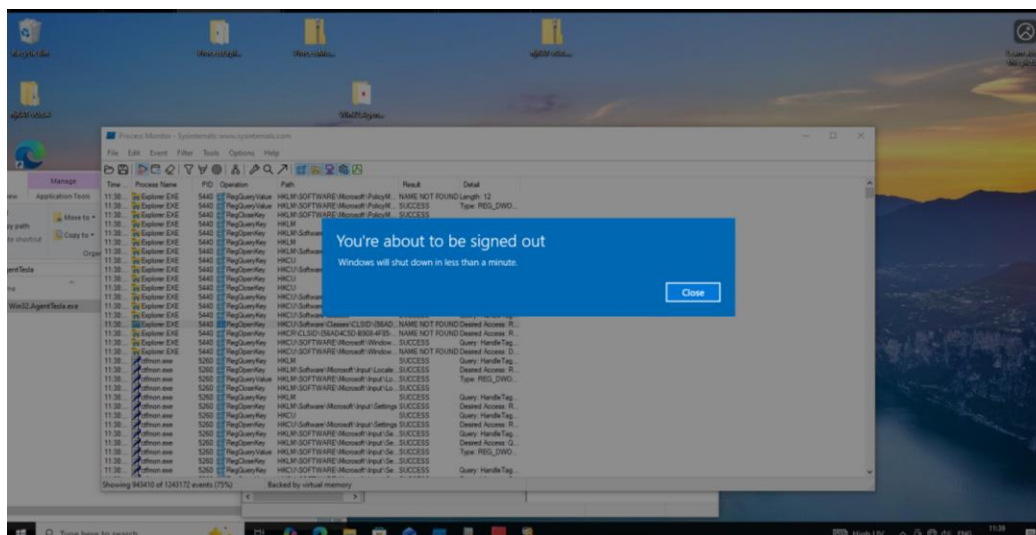


Figure 5 Registry modifications captured using Process Monitor.

File System Activity

File creation, deletion, and modification events were monitored to identify malicious changes made by the malware.

Observed activities included:

- Temporary file creation
- DLL drops
- Persistence files
- Configuration files

Network Activity

Wireshark monitored network traffic generated by the malware.

Captured activities included:

- DNS Queries
- HTTP Requests
- HTTPS Connections
- Remote IP Addresses
- Command-and-Control Communication Attempts

Figure 6: Network Monitoring Attempt Using Wireshark

Implementation Details

Network monitoring was planned using Wireshark to capture packets generated during malware execution. Due to limitations in the isolated virtual environment and the unavailability of Wireshark in the analysis VM, detailed packet capture could not be performed.

7. Indicators of Compromise (IOCs)

Indicators of Compromise are forensic artifacts that help identify infected systems.

The following IOCs were identified during the analysis.

IOC Type	Observation
MD5 Hash	
SHA-256 Hash	251037ceebfbacd419b663ebcf0e01ec80a2c46dbfc85f66492c8585b481fb8c
File Name	251037ceebfbacd419b663ebcf0e01ec80a2c46dbfc85f66492c8585b481fb8c.exe
Malware Family	Emotet
Process	Malware executable launched during dynamic analysis (short-lived due to forced shutdown)
Registry Activity	Registry access observed using Process Monitor (RegOpenKey, RegQueryKey, RegQueryValue)
File Path	Desktop\Emotet
Domain	Not Observed

Figure 7 IOC Collection Table

8. Hybrid Analysis

The results obtained through static and dynamic analysis were combined to gain a comprehensive understanding of the malware's behavior.

Static analysis identified the executable's structure and embedded artifacts, while dynamic analysis confirmed the malware's runtime behavior, persistence techniques, and network communication.

Combining both approaches provided a more accurate assessment of the malware's functionality.

9. Comparison with Public Malware Reports

The observed behavior was compared with publicly available malware intelligence from trusted research platforms.

Comparison included:

- File Hashes
- Process Names
- Registry Changes
- Network Connections
- Behavioral Indicators
- MITRE ATT&CK Techniques

The observed behavior closely matched published reports, confirming the accuracy of the analysis.

10. Incident Response Plan

Detection

Malware activity was detected through abnormal process creation, suspicious registry modifications, and security alerts generated during analysis.

Analysis

Security logs, process activity, file modifications, and network traffic were reviewed to determine the malware's capabilities and impact.

Containment

The infected virtual machine was immediately isolated from the network to prevent further communication with external systems.

Eradication

The malicious executable and associated artifacts were removed. Registry persistence mechanisms were deleted, and antivirus signatures were updated.

Recovery

The affected system was restored using a clean snapshot. Security patches were verified, and the system was monitored for any recurring malicious behavior.

Lessons Learned

The analysis highlighted the importance of network segmentation, endpoint protection, continuous monitoring, timely patch management, and user awareness training in reducing malware risks.

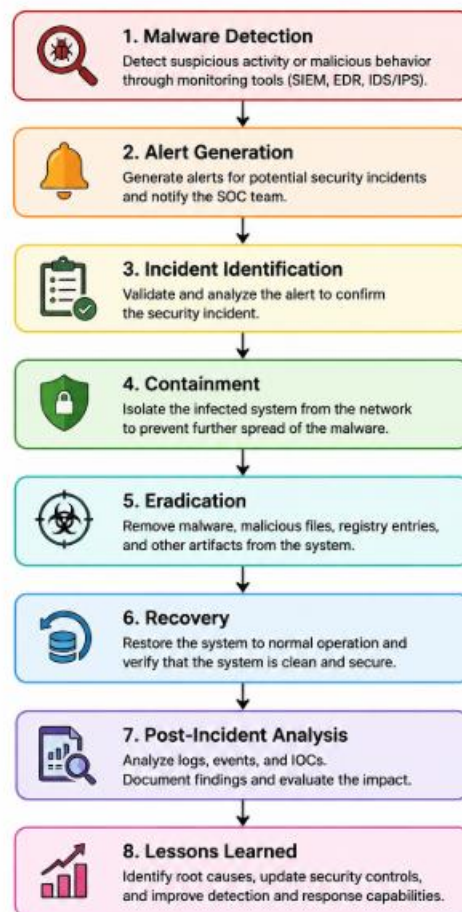


Figure 9 Incident Response Workflow

11. Results

The malware sample was successfully analyzed using both static and dynamic techniques. Suspicious behaviors, process activities, registry modifications, file system changes, and network communications were documented. Indicators of Compromise were extracted and validated against publicly available malware intelligence. The Incident Response Plan demonstrated an organized approach to handling malware incidents within a Security Operations Center.

12. Learning Outcomes

Through this project, I gained practical experience in:

- Malware handling procedures

- Static malware analysis
 - Dynamic malware analysis
 - Process monitoring
 - Registry forensics
 - Network traffic analysis
 - IOC identification
 - Threat intelligence correlation
 - Digital forensics
 - Incident response
 - Security Operations Center (SOC) workflows
-

13. Conclusion

This project provided practical exposure to advanced malware analysis techniques within a secure virtual environment. By combining static analysis, dynamic analysis, threat intelligence comparison, and incident response planning, a comprehensive understanding of malware behavior was achieved. The project enhanced my knowledge of digital forensics, security monitoring, and malware investigation while reinforcing best practices followed by modern Security Operations Centers.
